Basic Pandas

Example Data

```
data = {
    'user': ['Alice', 'Bob', 'Charlie', 'David', 'Emma', 'Frank', 'Grace', 'Henry', 'Iris', 'Jack',
             'Kelly', 'Liam', 'Mia', 'Noah', 'Olivia', 'Peter', 'Quinn', 'Rachel', 'Sam', 'Tina'],
    'province': ['Bangkok', 'Bangkok', 'Chiang Mai', 'Bangkok', 'Phuket',
                 'Chiang Mai', 'Bangkok', 'Phuket', 'Chiang Mai', 'Bangkok',
                 'Phuket', 'Bangkok', 'Chiang Mai', 'Bangkok', 'Phuket',
                 'Chiang Mai', 'Bangkok', 'Phuket', 'Chiang Mai', 'Bangkok'],
    'age': [25, 34, 28, 45, 31, 29, 52, 38, 27, 41,
            33, 26, 48, 35, 30, 44, 39, 28, 50, 32],
    'income': [35000, 48000, 32000, 65000, 42000,
               38000, 75000, 55000, 30000, 62000,
               45000, 36000, None, 58000, 47000,
               None, 68000, 43000, 52000, 40000],
    'spending': [25000, 35000, 28000, 45000, 38000,
                 32000, 55000, 48000, 25000, 50000,
                 40000, 30000, 42000, 48000, 42000,
                 38000, 52000, 39000, 45000, 35000],
    'debt': [50000, 120000, 0, 200000, 80000,
             30000, 150000, 95000, 0, 180000,
             100000, 45000, 70000, None, 88000,
             110000, 175000, 92000, 140000, 65000]
}
```

Example Data

```
(dsde) [redacted]Learning\python-pandas>python 01-self-learning.py
       user    province  age   income  spending      debt
0     Alice     Bangkok   25  35000.0     25000   50000.0
1       Bob     Bangkok   34  48000.0     35000  120000.0
2   Charlie  Chiang Mai   28  32000.0     28000       0.0
3     David     Bangkok   45  65000.0     45000  200000.0
4      Emma      Phuket   31  42000.0     38000   80000.0
5     Frank  Chiang Mai   29  38000.0     32000   30000.0
6     Grace     Bangkok   52  75000.0     55000  150000.0
7     Henry      Phuket   38  55000.0     48000   95000.0
8      Iris  Chiang Mai   27  30000.0     25000       0.0
9      Jack     Bangkok   41  62000.0     50000  180000.0
10    Kelly      Phuket   33  45000.0     40000  100000.0
11     Liam     Bangkok   26  36000.0     30000   45000.0
12      Mia  Chiang Mai   48      NaN     42000   70000.0
13     Noah     Bangkok   35  58000.0     48000       NaN
14   Olivia      Phuket   30  47000.0     42000   88000.0
15    Peter  Chiang Mai   44      NaN     38000  110000.0
16    Quinn     Bangkok   39  68000.0     52000  175000.0
17   Rachel      Phuket   28  43000.0     39000   92000.0
18      Sam  Chiang Mai   50  52000.0     45000  140000.0
19     Tina     Bangkok   32  40000.0     35000   65000.0
```

Example Data

```
(dsde) [redacted]Learning\python-pandas>python 01-self-learning.py
       user    province  age   income  spending      debt
0     Alice     Bangkok   25  35000.0     25000   50000.0
1       Bob     Bangkok   34  48000.0     35000  120000.0
2   Charlie  Chiang Mai   28  32000.0     28000       0.0
3     David     Bangkok   45  65000.0     45000  200000.0
4      Emma      Phuket   31  42000.0     38000   80000.0
5     Frank  Chiang Mai   29  38000.0     32000   30000.0
6     Grace     Bangkok   52  75000.0     55000  150000.0
7     Henry      Phuket   38  55000.0     48000   95000.0
8      Iris  Chiang Mai   27  30000.0     25000       0.0
9      Jack     Bangkok   41  62000.0     50000  180000.0
10    Kelly      Phuket   33  45000.0     40000  100000.0
11     Liam     Bangkok   26  36000.0     30000   45000.0
12      Mia  Chiang Mai   48      NaN     42000   70000.0
13     Noah     Bangkok   35  58000.0     48000       NaN
14   Olivia      Phuket   30  47000.0     42000   88000.0
15    Peter  Chiang Mai   44      NaN     38000  110000.0
16    Quinn     Bangkok   39  68000.0     52000  175000.0
17   Rachel      Phuket   28  43000.0     39000   92000.0
18      Sam  Chiang Mai   50  52000.0     45000  140000.0
19     Tina     Bangkok   32  40000.0     35000   65000.0
```

DataFrame - Infomation

2. ดู DataFrame คร่าวๆ

python

```python
df.shape       # กี่ row กี่ column
df.dtypes      # type ของแต่ละ column
df.head(2)     # ดู 2 row แรก
df.tail(2)     # ดู 2 row ท้าย
```

DataFrame – Column Selection

```python
# column operator
# -- select a row
print(df["user"])

# -- select many columns
subset = df[['user', 'age', 'income']]
print(subset.head())
```

```
0       Alice
1         Bob
2     Charlie
3       David
4        Emma
5       Frank
6       Grace
7       Henry
8        Iris
9        Jack
10      Kelly
11       Liam
12        Mia
13       Noah
14     Olivia
15      Peter
16      Quinn
17     Rachel
18        Sam
19       Tina
Name: user, dtype: object
      user  age   income
0    Alice   25  35000.0
1      Bob   34  48000.0
2  Charlie   28  32000.0
3    David   45  65000.0
4     Emma   31  42000.0
```

DataFrame – Row Selection

Start with True-False Array of each row. When an True-False array is put in the DataFrame it will select only the true row.

```python
# Row operator (Filter)
# Start With True-False Array of each row
condition = [True, False,
             True, False,
             True, False,
             True, False,
             True, False,
             True, False,
             True, False,
             True, False,
             True, False,
             True, False,
             True, False,
             True, False,]
print(df[condition])
```

	user	province	age	income	spending	debt
0	Alice	Bangkok	25	35000.0	25000	50000.0
2	Charlie	Chiang Mai	28	32000.0	28000	0.0
4	Emma	Phuket	31	42000.0	38000	80000.0
6	Grace	Bangkok	52	75000.0	55000	150000.0
8	Iris	Chiang Mai	27	30000.0	25000	0.0
10	Kelly	Phuket	33	45000.0	40000	100000.0
12	Mia	Chiang Mai	48	NaN	42000	70000.0
14	Olivia	Phuket	30	47000.0	42000	88000.0
16	Quinn	Bangkok	39	68000.0	52000	175000.0
18	Sam	Chiang Mai	50	52000.0	45000	140000.0

DataFrame – Row Selection

Define the condition (Who earns more than 50,000?)

```
condition = df['income'] > 50000
print(condition.head(5))
```

```
0    False
1    False
2    False
3     True
4    False
Name: income, dtype: bool
```

In the picture we show only the first 5 rows from all 20 rows by using .head(5).

DataFrame – Row Selection

Put the condition in the DataFrame and displayed. It will show only users who have income greater than 50,000

```
condition = df['income'] > 50000
print(df[condition])
```

```
     user    province  age   income  spending      debt
3   David     Bangkok   45  65000.0     45000  200000.0
6   Grace     Bangkok   52  75000.0     55000  150000.0
7   Henry      Phuket   38  55000.0     48000   95000.0
9    Jack     Bangkok   41  62000.0     50000  180000.0
13   Noah     Bangkok   35  58000.0     48000       NaN
16  Quinn     Bangkok   39  68000.0     52000  175000.0
18    Sam  Chiang Mai   50  52000.0     45000  140000.0
```

DataFrame – Row Selection

You can use AND (&) OR (|) NOT(~) operator in condition

```python

condition = (df['income'] > 40000) & (df['province'] == 'Bangkok')
condition = (df['province'] == 'Chiang Mai') | (df['province'] == 'Phuket')
condition = ((df['province'] == 'Bangkok') & (df['age'] > 30)) | (df['debt'] == 0)
condition = df['province'] == 'Bangkok'
print(df[~condition])
# Instead of (df['province'] == 'Bangkok') | (df['province'] == 'Phuket')
condition = df['province'].isin(['Bangkok', 'Phuket'])
condition = df['age'].between(30, 40)

```

DataFrame – Create Column

```
df["income_usd"] = df["income"] / 35
```

	user	province	age	income	spending	debt	income_usd
0						000.0	1000.000000
1						000.0	1371.428571
2						0.0	914.285714
3	David	Bangkok	45	65000.0	45000	200000.0	1857.142857
4	Emma	Phuket	31	42000.0	38000	80000.0	1200.000000
5	Frank	Chiang Mai	29	38000.0	32000	30000.0	1085.714286
6	Grace	Bangkok	52	75000.0	55000	150000.0	2142.857143
7	Henry	Phuket	38	55000.0	48000	95000.0	1571.428571
8	Iris	Chiang Mai	27	30000.0	25000	0.0	857.142857
9	Jack	Bangkok	41	62000.0	50000	180000.0	1771.428571
10	Kelly	Phuket	33	45000.0	40000	100000.0	1285.714286
11	Liam	Bangkok	26	36000.0	30000	45000.0	1028.571429
12	Mia	Chiang Mai	48	NaN	42000	70000.0	NaN
13	Noah	Bangkok	35	58000.0	48000	NaN	1657.142857
14	Olivia	Phuket	30	47000.0	42000	88000.0	1342.857143
15	Peter	Chiang Mai	44	NaN	38000	110000.0	NaN
16	Quinn	Bangkok	39	68000.0	52000	175000.0	1942.857143
17	Rachel	Phuket	28	43000.0	39000	92000.0	1228.571429
18	Sam	Chiang Mai	50	52000.0	45000	140000.0	1485.714286
19	Tina	Bangkok	32	40000.0	35000	65000.0	1142.857143

DataFrame – Sort

```python

df.sort_values("income").head(6)                    # lowest → highest
df.sort_values("income", ascending=False).head(6)   # highest → lowest

```

```
       user    province  age   income  spending     debt
8      Iris  Chiang Mai   27  30000.0     25000      0.0
2   Charlie  Chiang Mai   28  32000.0     28000      0.0
0     Alice     Bangkok   25  35000.0     25000  50000.0
11     Liam     Bangkok   26  36000.0     30000  45000.0
5     Frank  Chiang Mai   29  38000.0     32000  30000.0
19     Tina     Bangkok   32  40000.0     35000  65000.0
     user province  age   income  spending      debt
6   Grace  Bangkok   52  75000.0     55000  150000.0
16  Quinn  Bangkok   39  68000.0     52000  175000.0
3   David  Bangkok   45  65000.0     45000  200000.0
9    Jack  Bangkok   41  62000.0     50000  180000.0
13   Noah  Bangkok   35  58000.0     48000       NaN
7   Henry   Phuket   38  55000.0     48000   95000.0
```

DataFrame – Group By

```python
print(df.groupby("province")["income"].mean())
print(df.groupby("province")["income"].max())
```

```
province
Bangkok        54111.111111
Chiang Mai     38000.000000
Phuket         46400.000000
Name: income, dtype: float64
province
Bangkok        75000.0
Chiang Mai     52000.0
Phuket         55000.0
Name: income, dtype: float64
```

DataFrame – Group By

```python
print(df.groupby("province")["income"].mean())
print(df.groupby("province")["income"].max())
```

```
province
Bangkok        54111.111111
Chiang Mai     38000.000000
Phuket         46400.000000
Name: income, dtype: float64
province
Bangkok        75000.0
Chiang Mai     52000.0
Phuket         55000.0
Name: income, dtype: float64
```

Basic Aggregation

01 basic pandas practice

Example Data

```
import pandas as pd

data = {
    "name":       ["Alice", "Bob", "Charlie", "Diana", "Eve", "Frank"],
    "department": ["HR", "IT", "IT", "Finance", "HR", "Finance"],
    "salary":     [55000, 72000, 68000, 80000, 52000, 91000],
    "age":        [29, 34, 28, 41, 35, 47]
}

df = pd.DataFrame(data)
```

Basic Aggregation (whole column)

```python
df["salary"].sum()     # total
df["salary"].mean()    # average
df["salary"].min()     # minimum
df["salary"].max()     # maximum
df["salary"].count()   # count non-null
df["salary"].std()     # standard deviation
```

GROUP BY + single aggregate

```python
df.groupby("department")["salary"].mean()
df.groupby("department")["salary"].sum()
df.groupby("department")["salary"].count()
```

SQL equivalent → `SELECT department, AVG(salary) FROM employees GROUP BY department`

GROUP BY + multiple aggregates

```python
df.groupby("department").agg(
    avg_salary = ("salary", "mean"),
    max_salary = ("salary", "max"),
    headcount  = ("name",   "count")
)
```

SQL equivalent:

```sql
SELECT department,
       AVG(salary) AS avg_salary,
       MAX(salary) AS max_salary,
       COUNT(name) AS headcount
FROM employees
GROUP BY department
```

GROUP BY multiple columns

```python
df.groupby(["department", "age"])["salary"].mean()
```

SQL equivalent → `GROUP BY department, age`

HAVING (filter after group by)

```python
result = df.groupby("department")["salary"].mean()
result[result > 65000]   # only departments with avg salary > 65000
```

SQL equivalent → `HAVING AVG(salary) > 65000`

value_counts
(like COUNT + GROUP BY in one shot)

python

```python
df["department"].value_counts()
```

```
IT          2
HR          2
Finance     2
```

SQL equivalent → `SELECT department, COUNT(*) FROM employees GROUP BY department`

describe() — all stats at once

```python
df.groupby("department")["salary"].describe()
```

gives you count, mean, std, min, 25%, 50%, 75%, max — all in one table.